

Lab0 实验报告

实验任务 1

1. 认真阅读文档，学习 Git 的基本用法，并在报告中回答文档中的问题。（15 分）

需要回答的问题

- 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？
- 思考一下，Git 为什么要设计“暂存-提交”两个步骤？
- `git branch` 和 `git branch -a` 的区别是什么？查阅资料并回答。

1. 我此前没有正式的多人协同开发经历。在本次实验之前，仅独立完成代码编写，没有团队协作的版本管理经验。通过本次 Git 实验，我才了解多人开发需要通过版本控制工具来同步代码、解决冲突。

2. Git 分为工作区 → 暂存区（`git add`）→ 版本仓库（`git commit`）三层结构，设计两步的核心目的：

- 精细化挑选提交内容：工作区里可能同时修改了多个文件，有些改动是完成的功能，有些是临时调试代码。可以只把需要保存的改动加入暂存区，不必一次性提交全部修改，实现分批提交。
- 校验预览，保证提交质量：加入暂存区后，可以用 `git diff --cached` 查看暂存区内的修改，确认代码无误后再执行 `commit`，避免把测试垃圾、临时注释提交到版本历史。
- 聚合多次修改为一个逻辑快照：可以多次修改文件、多次 `git add` 更新暂存区，最后一次性 `commit`，把一组关联的改动打包成一条清晰的提交记录，让每一次 `commit` 都对应一个完整的功能变更。

3. answer:

- `git branch`：仅列出本地分支，也就是当前本机仓库创建的分支，不会显示远程仓库的分支。
- `git branch -a (--all)`：列出全部分支，包含本地分支 + 远程追踪分支（`remotes/origin/xxx`这类，是本地保存的远程仓库分支镜像，不能直接在上面编辑代码）。

实验任务 2

2. 使用我们的模板仓库建立个人仓库，完成 `main.c` 文件中的 TODO 部分并进行一次 `commit`。（50 分）

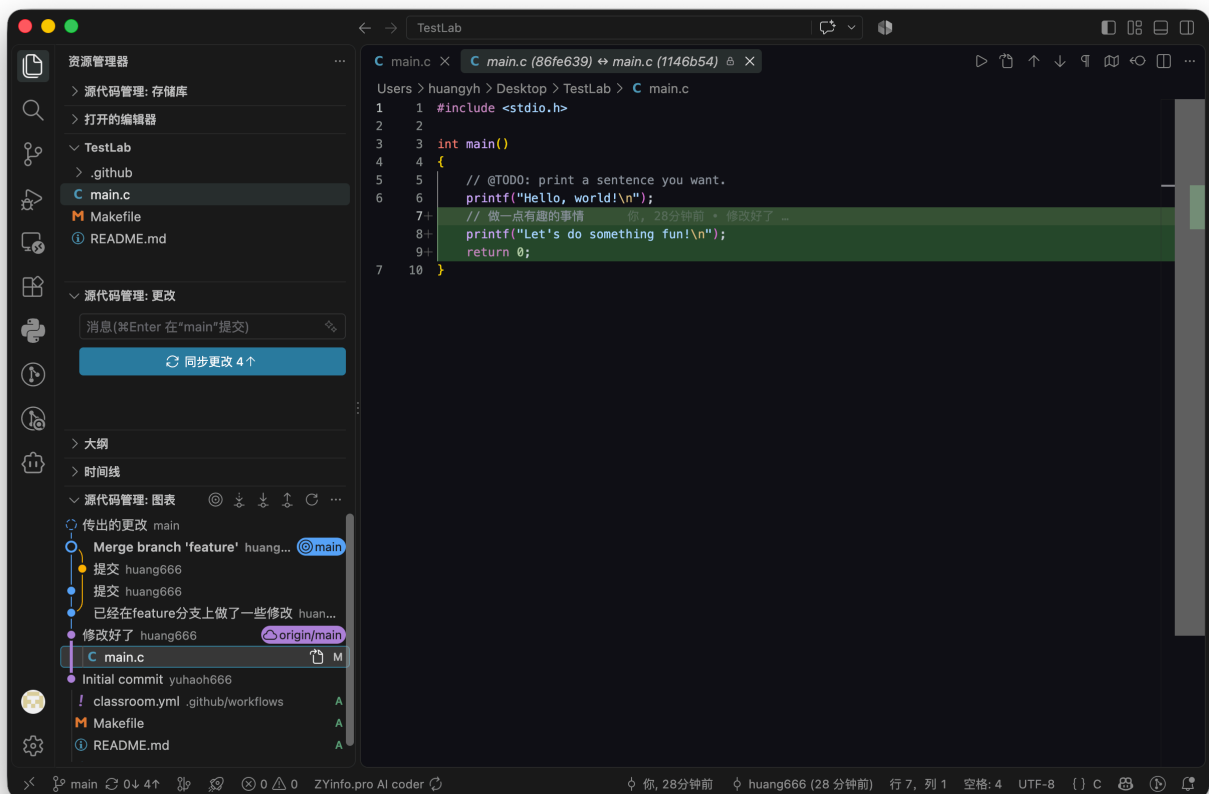
Info

只要填入任意字符串就算完成，当然你也可以随意发挥（程序的正确性不纳入计分，有修改即可）。
如果你想要编译运行 `main.c`，执行

```
make
./main
make clean
```

实验步骤：

1. 建立个人仓库
2. 在桌面新建一个文件夹，命名为TestLab
3. 输入命令：cd ~/Desktop/TestLab
4. 输入命令：git init
5. 复制仓库链接，通过终端命令连接远程仓库：git remote add origin <https://github.com/yuhaoh666/huang-s-lab.git>
6. 在vs code中修改代码，保存，提交
7. 输入命令：git push origin main



实验任务 3

3. 在下面的三个网页中任选其二进行阅读，简要概括其内容，并谈谈你对“为什么要学习 Git”这个问题的理解。（15 分）

- [Commit Message 规范](#)
- [Git Flow 分支控制](#)
- [语义化版本](#)

一、网页内容概括

1. Commit Message 规范

本文介绍 Git 提交信息的标准化 Angular 提交规范，一套规范的 commit message 分为 **Header (必填)**、**Body**、**Footer**三段。Header 包含改动类型 type、影响模块 scope、简短描述 subject；其中 type 区分 feat（新增功能）、fix（修复 bug）、docs（文档修改）、refactor（代码重构）等 7 大类。

规范提交信息有三大价值：快速查阅提交历史、筛选特定类型的代码改动、自动生成版本变更日志 CHANGELOG。文章同时介绍 commitizen、conventional-changelog 等工具，帮助自动生成、校验规范提交信息，统一团队提交风格。

2. Git Flow 分支控制

Git Flow 是一套成熟的 Git 分支协作模型，为软件开发定义标准化分支策略。它包含**2 个长期主分支**：master 保存随时可部署的稳定生产代码；develop 作为开发集成分支，存放所有正在开发的功能代码。另外有 3 类临时短分支：

- feature：从 develop 分出，开发新功能，完成后合并回 develop；
- release：版本发布前的准备分支，用于测试与微调；
- hotfix：从 master 分出，紧急修复线上故障，修复后同步合并到 master 和 develop。该工作流隔离了功能开发、版本发布、线上紧急修复，减少多人开发的代码冲突，保证生产分支稳定，同时配套 git-flow 工具简化分支操作。

二、对“为什么要学习 Git”的理解

Git 是现代软件开发的分布式版本控制工具，学习 Git 的意义不止于保存代码，而是管理项目的整个变更过程：

1. **版本追溯与安全**：Git 会记录每一次代码提交快照，代码写错、误删时可以随时回退到历史版本，避免代码丢失。配合规范的 commit message，能清晰看懂每一处修改的目的。
2. **支持并行协作开发**：Git 强大的分支能力是 Git Flow 这类工作流的基础。多人可以在独立分支上并行开发新功能、修复 bug，互不干扰，开发完成后再合并代码，极大降低团队协作成本。
3. **工程化管理**：Git 可以配合提交规范、分支策略、版本管理，自动生成变更文档，规范开发流程，方便项目迭代、版本发布与问题排查。
4. **分布式特性**：每个开发者本地都保存完整项目仓库，不依赖单一中心服务器，离线也可以提交记录，是目前几乎所有开源项目、企业项目的标准工具。
简单来说，Git 是代码的“时光机”和“协作平台”，是程序员必备的基础工程能力。

实验任务 4

4. 学习 Git 分支管理，新建 `feature` 分支，在该分支以及 `main` 分支上对 `main.c` 分别进行一次修改与提交（10 分）。

随后将 `feature` 分支 merge 到 `main` 分支（即切换回 `main` 分支执行 `git merge feature`），并处理发生的合并冲突（10 分）。

Important

在两个分支上的提交必须要满足：在 `main` 分支合并时会出现冲突。请你解决这个冲突，并在实验报告里截图表明你遇到并解决了冲突。

请阅读 `git merge` 部分，思考如何修改 `main.c` 会出现冲突。

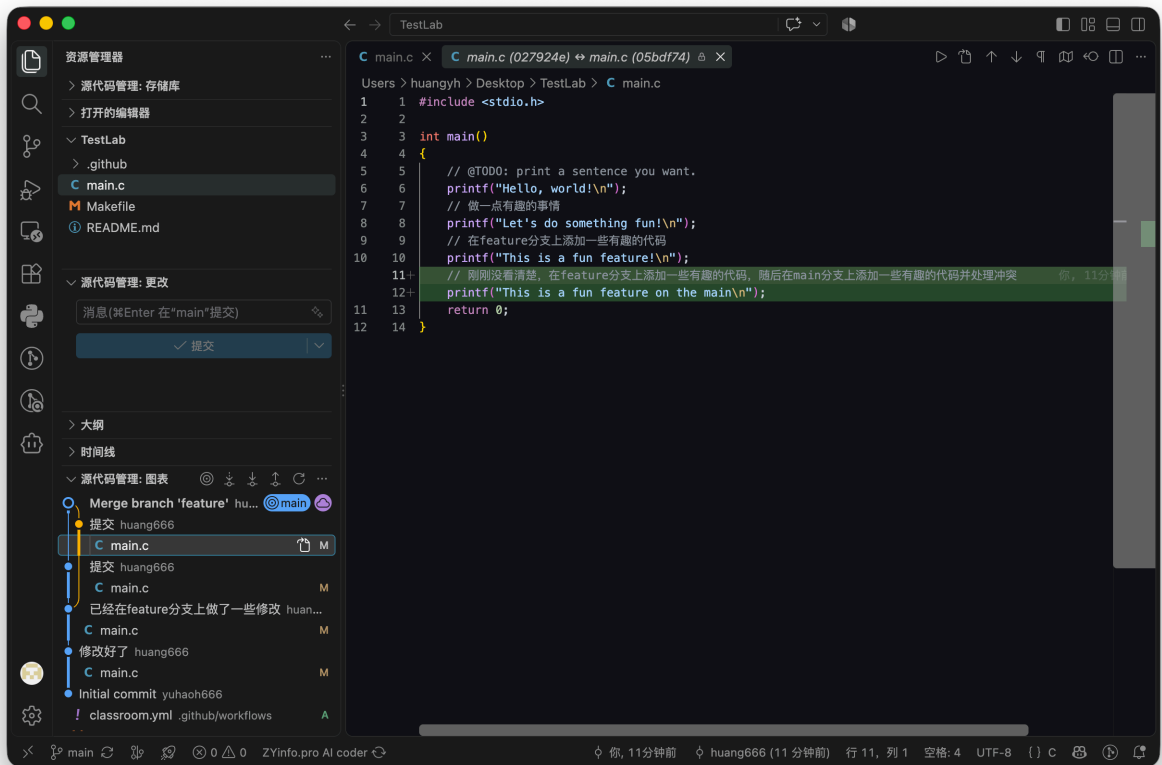
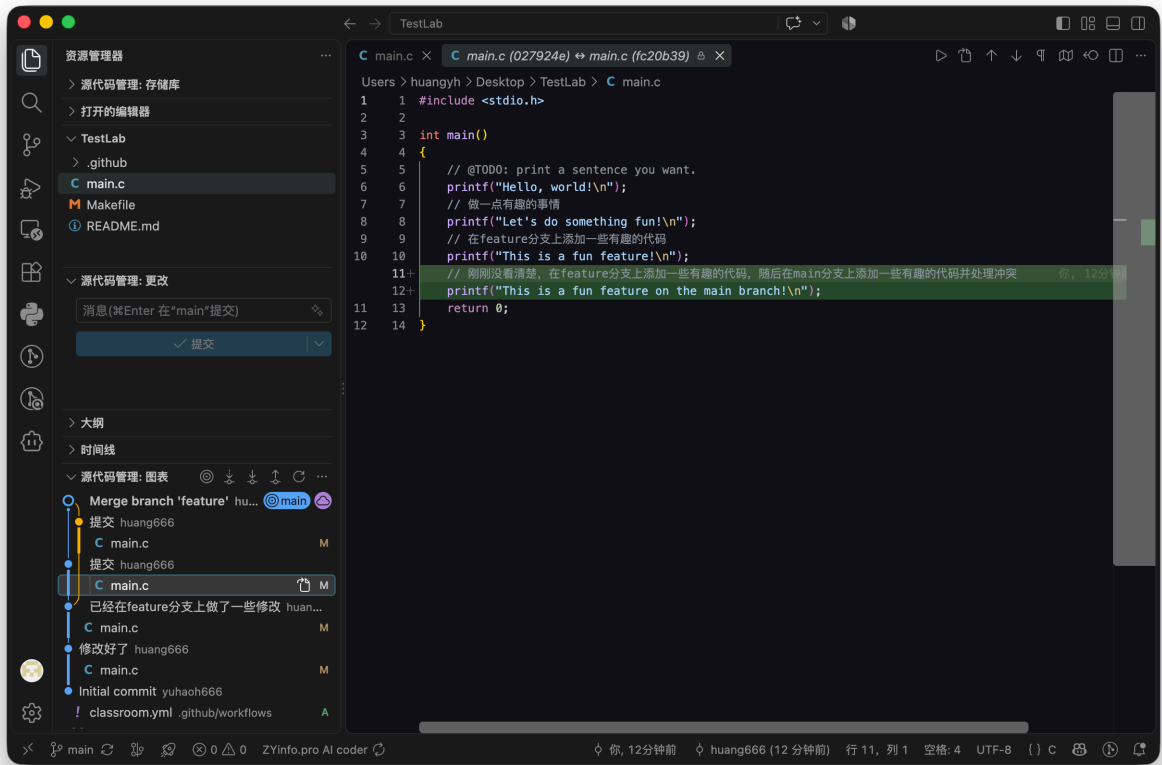
如果你两次提交之后合并没有出现冲突，不必担心，你可以不用撤回之前的提交，而是继续尝试提交修改并 merge，直到出现冲突并解决。

实验步骤：

1. 输入命令：`cd ~/Desktop/TestLab`
2. 在vs code上新建feature分支，当然也可以输入终端命令：`git branch feature`
3. 切换到分支feature：`git switch feature`
4. 在feature分支上修改main.c文件并提交
5. 切换到分支main：`git switch feature`
6. 在main分支上修改main.c文件并提交
7. 合并两个分支，输入命令：`git merge feature`
8. 冲突出现，按照给出提示返回vs code处理冲突
9. 输入命令：`git push origin main`，上传到GitHub

```
TestLab — -zsh — 80x24

Last login: Mon Sep 21 21:19:06 on ttys000
[huangyh@HuangdsdeMacBook-Pro ~ % cd ~/Desktop/TestLab ]
[huangyh@HuangdsdeMacBook-Pro TestLab % git switch feature ]
切换到分支 'feature'
[huangyh@HuangdsdeMacBook-Pro TestLab % git switch feature ]
已经位于 'feature'
[huangyh@HuangdsdeMacBook-Pro TestLab % git switch main ]
切换到分支 'main'
您的分支领先 'origin/main' 共 2 个提交。
（使用 "git push" 来发布您的本地提交）
[huangyh@HuangdsdeMacBook-Pro TestLab % git merge feature ]
自动合并 main.c
冲突（内容）：合并冲突于 main.c
自动合并失败，修正冲突然后提交修正的结果。
[huangyh@HuangdsdeMacBook-Pro TestLab % git push origin main ]
枚举对象中：12，完成。
对象计数中：100% (12/12)，完成。
使用 8 个线程进行压缩
压缩对象中：100% (10/10)，完成。
写入对象中：100% (10/10)，1.18 KiB | 1.18 MiB/s，完成。
总共 10（差异 6），复用 0（差异 0），包复用 0（来自 0 个包）
remote: Resolving deltas: 100% (6/6)，completed with 1 local object.
To https://github.com/yuhaoh666/huang-s-lab.git
1146b54..cfd0380 main -> main
```



实验任务 5

提交报告

5. 在 `main` 分支提交一份实验报告（实验报告单独评分），格式要求为 `PDF` 或 `Markdown`。内容包括：

- 文档中要求回答的问题
- 你的实验步骤
- 必要的截图
- 你的建议（可选）

